

Lab Electronic Systems: GP1:Distance controller

1st Ziad Elzein
Cairo, Egypt
Zelzein@stud.hs-heilbronn.de

2nd Mohd Omar Mohd EL Tallaa
Amman, Jordan
meltallaa@stud.hs-heilbronn.de

3rd Sybren Veninga
Hattemerbroek, Netherlands
Sveninga@stud.hs-heilbronn.de

4th Zaid Alkhair Mohammed Yousef BaniYounes
Amman, Jordan
zyounes@stud.hs-heilbronn.de

Abstract—Autonomous mobile robots are often employed in numerous applications that require safe movement without human intervention. For this particular project, we have designed and constructed a mobile robot which consists of a Raspberry Pi 4 computer running on Ubuntu 22.04 and ROS 2 Humble. The robot uses an HC-SR04 ultrasonic sensor to detect objects, a LiDAR sensor to scan the environment, and an L298N motor driver to operate two DC motors. All of the software was written in the form of multiple ROS 2 nodes that enable intercommunication between various pieces of hardware. Initially, the goal of our project was to build a robot capable of following an object while maintaining a safe distance from it. Due to technical problems with the LiDAR sensor, however, the final implementation of the robot became an obstacle avoidance system that relied mostly on ultrasonic readings.

Index Terms—ROS 2, Raspberry Pi 4, Mobile Robot, LiDAR, Ultrasonic Sensor, Obstacle Avoidance.

I. INTRODUCTION AND PROBLEM STATEMENT

There has been a rapid growth in the use of Autonomous Mobile Robots (AMRs) over the last ten years, moving from being used in highly controlled manufacturing plants to more unpredictable and dynamic locations such as hospitals, retail shops, and even homes. Contrary to the conventional Automated Guided Vehicles (AGVs), which use tangible infrastructure, such as magnetic tapes and wires, AMRs sense their environment dynamically. AMRs collect data from the sensors in their immediate environment, analyze them using middleware, and undertake navigation without any human intervention.

The key purpose of the capstone project was to develop a small-size, highly responsive mobile autonomous robot based on a Raspberry Pi 4 computer together with Robot Operating System 2 (ROS 2). Initially, we had a very broad scope of work, that is, to build the system able to detect a certain object or individual, calculate relative vector, and control the robot motors by changing the speed in order to follow the object, while keeping safety margin. It meant that we had to use the technique of sensor fusion, which would consist in combining the high frequency data about distance from an ultrasonic sensor together with location data from LiDAR.

In order to ensure that our software was scalable and easy to debug, we made use of the ROS 2 architecture. Instead of

creating a large single file containing all our high and low level functions for our software, we decided to modularize the software. This way, the entire software was organized into nodes which only did one thing: drive sensors, filter signals, control kinematics, or drive hardware actuators. This setup meant that we could control the variables; for example, we could check our ultrasonic sensor's data using the terminal.

However, taking it from simulation to actual hardware posed some very difficult technical challenges. Although our ultrasonic sensors and DC motor controllers were working fine and up to expectation, incorporating the LiDAR sensor became very unpredictable. The hardware was working correctly and was producing laser scans arrays but running those arrays on an object tracking algorithm proved to be very difficult in terms of our processing system and hardware reliability. It took us a few weeks to fix the communication and node sync errors. Due to time constraints in academic settings, the object following feature could not become reality.

Instead of scrapping the design, however, we flipped the primary mission of the robot. We offered the current modular design to perform strong continuous obstacle avoidance. The new design will allow the robot to move autonomously using an exploration mode in which it is constantly checking the forward ultrasonic sensor. In case an object exceeds the preset safe distance, the robot stops moving, does a random rotation maneuver, and continues exploring. This change allowed us to test our own ROS 2 design and prove its flawless operation.

The second major difficulty, though no less significant than the previous one, was the lack of our skills at the beginning of the project. None of us had any prior practical experience with ROS 2, Linux networking, and robotics in general. All our milestones ranging from the implementation of rules for USB devices to tuning the PWM frequencies on an H-bridge had to be researched and implemented anew. In retrospect, going through this learning process and discovering how to transform a broken system into a working prototype was the most important lesson that we learned as engineers.

II. TECHNICAL BACKGROUND

a) Robot Operating System (ROS 2): Contrary to what the acronym implies, ROS is not a conventional operating

system such as Windows or macOS. Instead, ROS is a set of open-source middleware libraries and tools aimed at easing the process of creating highly effective software for robots. The chosen version of ROS for the current project is Humble Hawksbill. This is the LTS release compatible with Ubuntu 22.04 LTS installed on our Raspberry Pi.

The key element in ROS 2 is that it uses the DDS standard for internal communications. This enables the division of a program into separate "nodes." These nodes communicate with each other through a pub/sub communication model on topics (channels). For instance, our ultrasonic node will publish the distance values to a topic while our PID node only subscribes to that topic without knowing anything about how the distance values were calculated by the ultrasonic node. That is the decoupling that makes ROS 2 so useful in teams since even if one node fails due to a bug, the entire DDS system keeps working.

In all stages of development, we made extensive use of ROS 2 command line interface (CLI). CLI commands like `ros2 topic echo`, `ros2 node info`, and `RQT_graph` enabled us to view real time flow of information and were very helpful in ensuring that our hardware drivers actually transmitted any sensible data before attaching the motor.

b) Differential Drive Kinematics: The chassis is designed using differential drive mechanism. In this system, there are two wheels that are driven separately from an axis while there are caster wheels which provide balance. The system allows for control of speeds of both motors in order to achieve full plane movement, even zero degree turnings.

The kinematic behavior of a differential drive robot is governed by the rotational speeds of its wheels. Let v represent the linear velocity of the robot, ω represent the angular velocity, r represent the radius of the wheels, and L represent the distance between the two wheels (the wheelbase). If w_R and w_L are the angular velocities of the right and left wheels respectively, the forward kinematics are defined by:

$$v = \frac{r(w_R + w_L)}{2}$$

$$\omega = \frac{r(w_R - w_L)}{L}$$

If both motors rotate at the same velocity ($w_R = w_L$), the angular velocity ω becomes zero, resulting in straight-line motion. If the motors spin in opposite directions at equal speeds ($w_R = -w_L$), the linear velocity v becomes zero, and the robot rotates perfectly on the spot. This mathematical simplicity makes differential drive highly reliable for small-scale embedded robotics.

c) Embedded Operating System Environment: The brain of the robot is the Raspberry Pi 4 Model B running the Ubuntu 22.04 LTS operating system. The Raspberry Pi 4 Model B is not like the lightweight microcontroller board (like an Arduino). Instead, it is a full computer running a Linux kernel that is required to handle the DDS traffic from ROS 2.

The setup of Pi was the first challenge we faced. After flashing the operating system, it was crucial for us to install the base packages of ROS 2 Humble, and dependency management became very important. Very often we would encounter a problem where building our customized ROS 2 workspace using `colcon build` would not work because there would be some missing Python dependencies or problems with our `CMakeLists.txt` file. We discovered that we could use `colcon build Simulink-install`, so that we wouldn't have to compile everything whenever we edited our Python nodes.

d) LiDAR Sensor Mechanics and Limitations: LiDAR uses Light Detection and Ranging to determine the distance through a laser that is used to illuminate the object and the reflected light is measured by a sensor. The time taken by the laser beam gives the accurate distance value. The 2D LiDAR rotates very fast, taking hundreds of distance readings each second, giving a data set known as `sensor_msgs/LaserScan` in ROS.

The plan was to create a map of the immediate environment of the robot using LiDAR, and enable it to track a mobile object. Making the sensor work was not an easy task. Normally used drivers did not work, and the professor had to provide a special package for it. After compilation, the sensor started spinning, but problems arose because of the known Linux problem of USB port assignment. Each time when the Pi reboots, the operating system assigns the sensor to some other port.

Additionally, we soon found out that handling the entire 360-degree sweep of several hundred floating point values several dozen times per second overwhelmed our network. We optimized the node by breaking the Python array and extracting only the array indices that represent the first 90 degrees of the robot's vision. In order to avoid further network queuing problems, we also changed the QoS quality of service level in ROS 2 from "Reliable" to "Best Effort," which made the network drop the packets instead of queuing them.

e) Ultrasonic Distance Sensing: The HC-SR04 ultrasonic sensor was the primary sensor used for detecting obstacles. The ultrasonic wave travels through the air and reflects off the object and then measures the time taken to detect an echo from the reflected waves to find the distance from the object.

The distance is determined using the following equation:

f)

$$d = \frac{v \times t}{2}$$

: where:

g) d is the measured distance,;

h) v is the speed of sound (approximately 343 m/s),;

i) t is the measured echo time.: The measured time is divided by two because the sound wave travels from the sensor to the object and back again.

The choice of the HC-SR04 ultrasonic sensor was due to its cheapness, good connectivity with the Raspberry Pi, and sufficient accuracy in measuring short distances. The distance in front of the robot was measured by the ultrasonic sensor in a continuous manner during the operation of our project.

This data was subsequently sent via the ROS 2 topic and then filtered by the controller.

j) **PID Control Algorithms:** To prevent the robot from executing jerky, aggressive movements, we implemented a Proportional-Integral-Derivative (PID) controller. A PID controller continuously calculates an error value $e(t)$ as the difference between a desired setpoint and a measured process variable, applying a correction based on proportional, integral, and derivative terms.

The continuous mathematical form of the control output $u(t)$ is:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Because our ROS 2 system operates discretely (processing data in loops), we implemented the discrete equivalent in our Python node:

$$u(k) = K_p e(k) + K_i \sum_{i=0}^k e(i) + K_d \frac{e(k) - e(k-1)}{dt}$$

Tuning the K_p , K_i , and K_d gains was an exercise in patience. If the proportional gain was too high, the robot oscillated violently left and right. If the derivative gain was too low, it failed to react fast enough to sudden obstacles.

k) **Signal Filtering:** It turns out that trying to use the raw HC-SR04 readings as input for the PID was an absolute mess, with single frame abnormalities causing massive jitters. Applying the typical moving average filter didn't help either since just one anomaly could distort the average of a 5 reading window significantly.

To resolve this, we engineered a median filter node. This node stores the last N measurements in a rolling buffer, sorts them sequentially, and outputs the middle value.

Should we have used our buffer to give us such measurements in sequence like [15, 14, 800, 16, 15], then using a straightforward average would provide us with an incorrect average of 172cm. Our median filter is used to sort this into [14, 15, 15, 16, 800] and provides us with the central value of 15cm.

III. SYSTEM DESIGN

The goal of our work was to create a mobile robot capable of moving independently and identifying any obstacles lying ahead. For that purpose, various hardware parts have been used along with the software structure that is based on ROS 2. Each piece of hardware serves a particular purpose, and all together make it possible for the robot to gather and process data as well as move.

The Raspberry Pi 4 is the main controller of the system. This controller gets information from the sensors and then processes that information using various nodes of ROS 2 and then passes the command to the motor driver. Figure X represents the overall architecture of the robot.

IV. MODULAR SYSTEMS DESIGN

Before starting the implementation, we designed the overall system architecture and defined how data would flow through the robot. The whole idea of having one big block of code for embedded robotics is really hard to debug, since any kind of malfunction while reading from the sensors would cause a total shutdown of the motors.

By strictly adhering to ROS 2's publisher/subscriber paradigm, we enforced separation of concerns. The overall processing pipeline follows a rigid chronology:

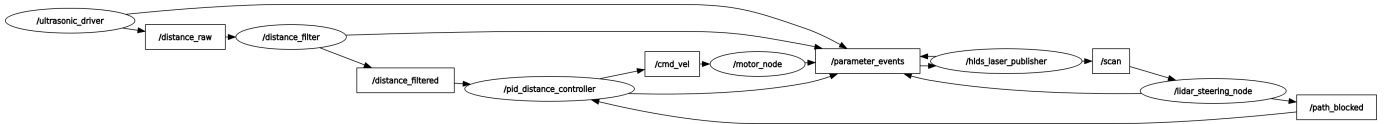
- a) **Hardware Polling:** Extract physical data from the environment.:
- b) **Sanitization:** Clean and format the raw data arrays.:
- c) **Kinematic Calculation:** Determine the necessary reaction based on the cleaned data.:
- d) **Hardware Actuation:** Translate the mathematical reaction into electrical PWM signals.:

V. NODE BREAKDOWN AND TOPIC LAYOUT

The node graph below visualizes the active Data Distribution Service (DDS) network topology of our robot. It clearly illustrates the separation of concerns within our publisher-subscriber architecture. The ultrasonic data pipeline begins with the /ultrasonic_driver publishing to /distance_raw, which is sanitized by the /distance_filter and passed to the /pid_distance_controller via /distance_filtered. Simultaneously, the LiDAR pipeline flows from the /hlds_laser_publisher through the /scan topic to the /lidar_steering_node, which outputs collision flags to /path_blocked. The /pid_distance_controller acts as the central hub, synthesizing these inputs to publish final velocity commands to the /cmd_vel topic, which are then translated into hardware action by the /motor_node. All active nodes are also shown subscribing to /parameter_events for dynamic configuration updates.

Specifically, the software ecosystem is strictly modular, dividing the system's core responsibilities across the following ROS 2 nodes:

- a) **Ultrasonic Driver Node (/ultrasonic_driver):** Directly interfaces with the GPIO pins to trigger the HC-SR04 sensor; calculates the acoustic time-of-flight, and publishes the raw distance measurements as std_msgs/msg/Float32 messages to the /distance_raw topic.:
- b) **Distance Filter Node (/distance_filter):** Subscribes to the /distance_raw topic, applies a 3-sample rolling median filter to eliminate high-frequency noise and environmental anomalies, and publishes the sanitized data to the /distance_filtered topic.:
- c) **PID Controller Node (/pid_distance_controller):** Functions as the central decision-making unit (the "brain" of the system). It subscribes to the /distance_filtered and /path_blocked topics. If the distance drops below the hardcoded 30 cm safety threshold, the PID algorithm kicks in to generate corrective angular and linear velocity commands. These are published as geometry_msgs/msg/Twist messages to the /cmd_vel topic.:



d) **LiDAR Steering Node (/lidar_steering_node):** Subscribes to the raw /scan hardware topic. It slices the data array to isolate a front-facing 90-degree cone and publishes a simple collision flag to the /path_blocked topic if a dense cluster of points breaches the established safety radius.:

e) **Motor Command Node (/motor_node):** Subscribes to the /cmd_vel topic and translates the mathematical velocity vectors from the PID controller into raw PWM duty cycles (0–100%). It then physically drives the L298N H-bridge via Raspberry Pi’s GPIO outputs.:

VI. IMPLEMENTATION

After completing the system design, the next step was to build and test the robot. Since the project combined hardware and software, we first tested every component individually before integrating them into one complete system. This approach helped us identify problems early and made debugging much easier.

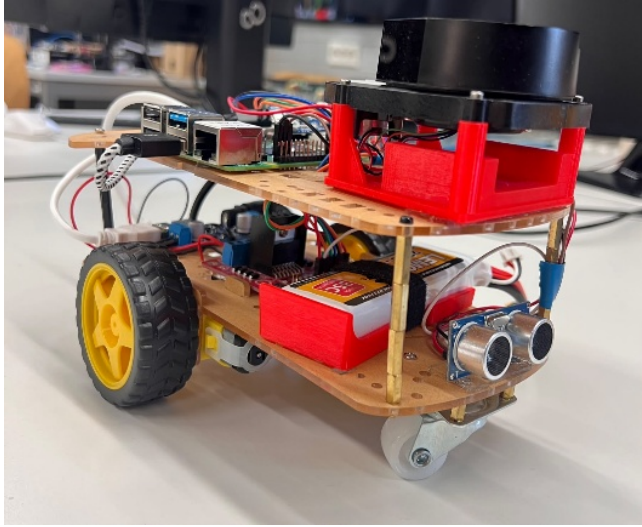


Fig. 2. The Final Design of the +Project

A. Mechanical Design and Manufacturing (Ziad)

The chassis required a custom design to securely house the Raspberry Pi, the bulky battery pack, the L298N motor driver, and the dual sensors, all while maintaining a balanced center of gravity to prevent tipping during aggressive maneuvers. I designed the chassis utilizing

SolidWorks. To facilitate rapid prototyping, the chassis was printed using PLA filament on a Raise3D N2 3D printer.

Soon enough, the issues of 3D printing tolerances came to light. The screw hole clearance holes on the mounting standoffs for the Raspberry Pi were off because of the plastic’s thermal shrinkage during printing. I had to revise the CAD file, and

this time I manually increased the clearance holes by 0.5mm. Moreover, there were also cable routing channels built right into the chassis floor so that no jumper wire would catch on the spinning wheels.

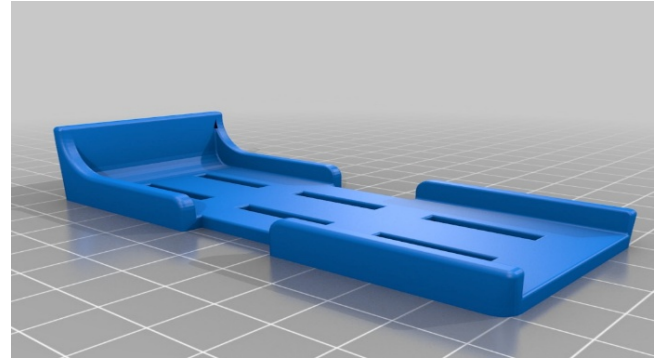


Fig. 1. 3D printed Lipo battery mount

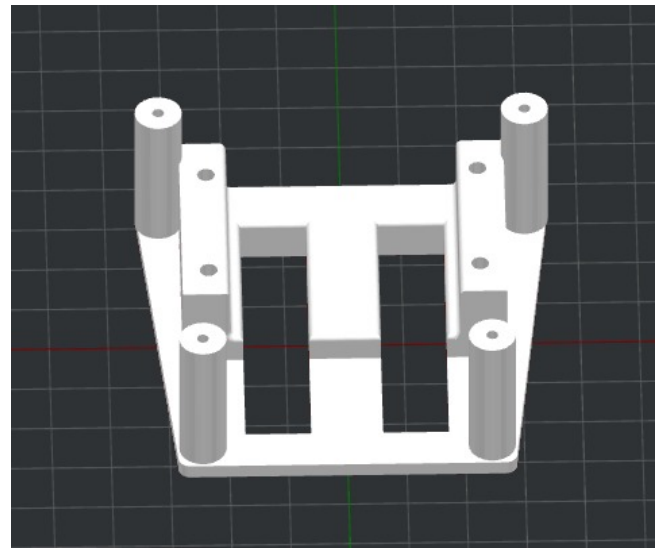


Fig. 2. 3D printed Lipo battery mount

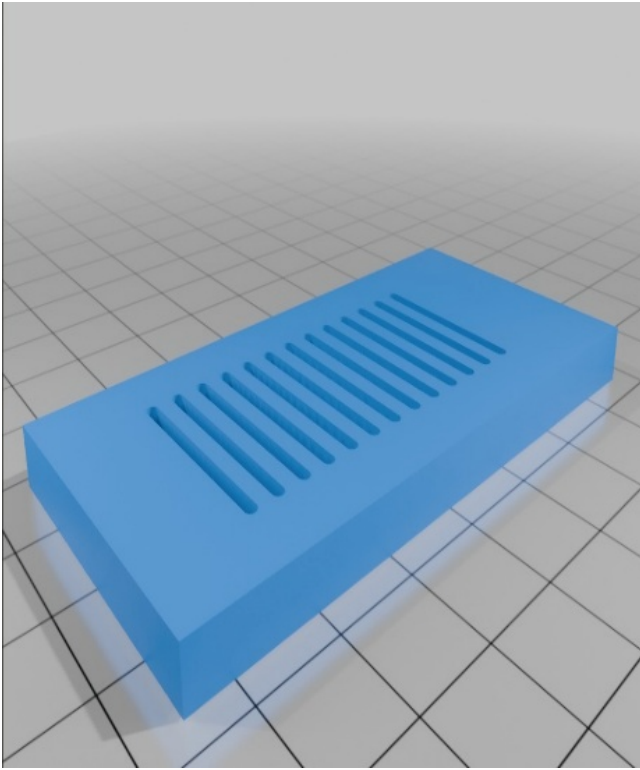


Fig. 5. 3D printed DC-DC Converter holder

B. Software Environment Configuration (Mohammed and Ziad)

This included the installation of Ubuntu 22.04 on the Raspberry Pi 4 and the installation of ROS 2 Humble. After that, a ROS 2 workspace was set up, where all the project packages were developed and compiled.

Each software piece was designed as a separate ROS 2 node. This approach simplified the organization of the whole project and enabled team members to work separately until the integration of all packages was completed.

After setting up the workspace, all packages were built through colcon build, and then the ROS environment was sourced before performing tests on the nodes. During the project development process, all new nodes were tested separately from the rest of the software.

C. Motor Control Node (Mohammed and Ziad)

The motor control block receives movement instructions from the controller and translates them to GPIO signals for the L298N motor driver. Based on the instructions it receives, the robot can move forwards, stay still, move left, or move right.

The motor driver controls each of the two DC motors individually, which enables differential steering. During testing, the PWM values were changed in order to make the robot move smoothly and to eliminate motor vibrations. There have been multiple changes done on the GPIO pins due to the initial incorrect rotation of motors.

After these adjustments, the robot was able to move reliably and respond correctly to commands generated by the controller.

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
import lgpio
import sys

class MotorNode(Node):
    def __init__(self):
        super().__init__('motor_node')

        # Open gpiochip0 (Confirmed by your 'ls /dev/gpiochip
        # *' command)
        try:
            self.chip = lgpio.gpiochip_open(0)
        except Exception as e:
            self.get_logger().error(f"Could not open gpiochip0:
            {e}")
            sys.exit(1)

        # Pinout mapping
        # ENA/ENB are PWM. IN1-IN4 are Direction pins.
        self.pins = {
            'L_PWM': 18, 'L_IN1': 17, 'L_IN2': 27,
            'R_PWM': 23, 'R_IN1': 22, 'R_IN2': 24
        }

        # --- Dead-zone compensation ---
        # MIN_DUTY: the lowest duty (%) that reliably starts
        # your motors FROM REST.
        # Below this they just buzz and stall. TUNE THIS for
        # your motors + battery:
        # lower it until a turn stalls, then nudge it back up
        # a few points.
        self.MIN_DUTY = 45.0
        # DEADBAND: |speed| below this is treated as a true
        # stop (no creeping).
        self.DEBAND = 0.02

        # Claim all pins as outputs
        for pin in self.pins.values():
            lgpio.gpio_claim_output(self.chip, pin)

        # Subscriber for velocity commands
        self.subscription = self.create_subscription(
            Twist,
            '/cmd_vel',
            self.listener_callback,
            10)

        self.get_logger().info("Motor Node Ready. Dead-zone
        floor active.")

    def set_motor(self, pwm_pin, in1, in2, speed):
        # Limit speed between -1.0 and 1.0
        speed = max(min(speed, 1.0), -1.0)

        # True stop: cut PWM and both direction pins low (
        # coast)
        if abs(speed) < self.DEBAND:
            lgpio.tx_pwm(self.chip, pwm_pin, 500, 0)
            lgpio.gpio_write(self.chip, in1, 0)
            lgpio.gpio_write(self.chip, in2, 0)
            return

        # Map |speed| in (0, 1] onto [MIN_DUTY, 100] so even a
        # small command
        # delivers enough torque to break static friction and
        # actually start.
        duty = self.MIN_DUTY + (100.0 - self.MIN_DUTY) * abs(
            speed)
        duty = min(duty, 100.0)

        # PWM at 500Hz for better motor startup torque
        lgpio.tx_pwm(self.chip, pwm_pin, 500, duty)

        # DIRECTION LOGIC (Flipped 0/1 to fix the 'moving
        # backward' issue)
        if speed > 0:
            lgpio.gpio_write(self.chip, in1, 0)
            lgpio.gpio_write(self.chip, in2, 1)
        else:
            lgpio.gpio_write(self.chip, in1, 1)
```



```

        lgpio.gpio_write(self.chip, in2, 0)

def listener_callback(self, msg):
    linear = msg.linear.x
    angular = msg.angular.z

    # "linear" drives both wheels forward; "angular"
    # rotates them oppositely.
    left_speed = linear + angular
    right_speed = linear - angular

    self.set_motor(self.pins['L_PWM'], self.pins['L_IN1'],
                    self.pins['L_IN2'], left_speed)
    self.set_motor(self.pins['R_PWM'], self.pins['R_IN1'],
                    self.pins['R_IN2'], right_speed)

def main(args=None):
    rclpy.init(args=args)
    node = MotorNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        # Emergency Stop on Ctrl+C
        node.set_motor(node.pins['L_PWM'], node.pins['L_IN1'],
                        node.pins['L_IN2'], 0)
        node.set_motor(node.pins['R_PWM'], node.pins['R_IN1'],
                        node.pins['R_IN2'], 0)
        node.get_logger().info("Motors Stopped.")

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

The motor control node receives the velocity command and produces the individual wheel speed for the left and right wheels by applying the principles of differential drive. It sends the necessary PWM signal and direction signal from the GPIO pins for controlling the motor driver such that the robot can travel forward, backward, or rotate while retaining the necessary distance from the target position.

VII. D) ULTRASONIC POLLING ARCHITECTURE (SYBREN)

Since the physical hardware was already occupied with the mechanical assembly in its early stages, the Ultrasonic Driver node was developed through simulation by creating a dummy data input in C++. This helped to test the structure of the ROS 2 topics in advance.

Once the hardware was wired, I rewrote the node into Python to natively utilize the RPi.GPIO library. During this transition, we hit a frustrating snag: the system kept running the old C++ dummy node instead of the new Python script. After hours of debugging the code itself, I discovered the issue lay in the ROS 2 build system. The colcon build tool had caught the old executable. Clearing the install and build directories and entirely sourcing the setup.bash file finally pushed the Python node to the live environment.

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
import time
import lgpio
import sys

class UltrasonicDriverNode(Node):
    def __init__(self):
        super().__init__('ultrasonic_driver')
        self.publisher_ = self.create_publisher(Float32, '
            distance_raw', 10)

```

```

self.timer = self.create_timer(0.04, self.
    measure_distance) # 25Hz

# Open gpiochip0 to share access cleanly with your
# motor node
try:
    self.chip = lgpio.gpiochip_open(0)
except Exception as e:
    self.get_logger().error(f"Could not open gpiochip0:
        {e}")
    sys.exit(1)

# GPIO Pins Setup (Your chosen pins)
self.TRIG = 19
self.ECHO = 26

lgpio.gpio_claim_output(self.chip, self.TRIG)
lgpio.gpio_claim_input(self.chip, self.ECHO)

# Ensure trigger starts low
lgpio.gpio_write(self.chip, self.TRIG, 0)
time.sleep(0.1)
self.get_logger().info("Ultrasonic LGPIO Driver
    Started.")

def measure_distance(self):
    # Trigger pulse (10 microseconds)
    lgpio.gpio_write(self.chip, self.TRIG, 1)
    time.sleep(0.00001)
    lgpio.gpio_write(self.chip, self.TRIG, 0)

    start_time = time.time()
    timeout = start_time + 0.04 # ~40ms max round trip
    safeguard

    # Wait for echo pulse start
    while lgpio.gpio_read(self.chip, self.ECHO) == 0:
        start_time = time.time()
        if start_time > timeout:
            self.get_logger().error("Sensor Timeout: Echo
                start not detected")
            return

    # Wait for echo pulse end
    stop_time = time.time()
    while lgpio.gpio_read(self.chip, self.ECHO) == 1:
        stop_time = time.time()
        if stop_time > timeout:
            break # Sensor range exceeded

    duration = stop_time - start_time
    # Speed of sound is 34300 cm/s. Divide by 2 for round-
    # trip path.
    distance = (duration * 34300.0) / 2.0

    msg = Float32()
    msg.data = float(distance)
    self.publisher_.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = UltrasonicDriverNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

This node interfaces directly with the ultrasonic sensor. It periodically sends a 10 μ s trigger pulse and measures the time until the echo is received. Using the measured round-trip travel time and the speed of sound, it calculates the distance to the object in front of the robot. The measured distance is published on the /distance_raw topic.

Distance Filter (Median Filter) (Sybren)

The HC-SR04 ultrasonic sensor can generate noise or erroneous distance readings because of factors like reflections, interference, and constraints in ultrasonic measurement. Because the control system needs an accurate distance value, a special Distance Filter node was designed to increase the accuracy of the distance value before passing it into the PID controller.

The node subscribes to the `/distance_raw` topic published by the Ultrasonic Driver and publishes the processed measurements on the `/distance_filtered` topic. Before filtering, each received measurement is validated to ensure that it lies within the operating range of the HC-SR04 sensor. Measurements outside the range of 3 cm to 380 cm are considered invalid and are replaced with a value of 0.0 cm.

For removal of peaks from the measurement, a median filter using a window size of three was applied. The new measurements are placed in a buffer, and the old measurements are discarded. Then the three values in the buffer are sorted, and the median is chosen as the filtered value. This type of filter has an advantage over the moving average filter, as it eliminates outliers but keeps the change in measurement intact.

The filtered measurement is finally published on the `/distance_filtered` topic, where it is used by the PID controller to generate stable movement commands. During testing, this approach significantly reduced sensor noise and improved the overall stability of the robot's movement.

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32

class DistanceFilterNode(Node):
    def __init__(self):
        super().__init__('distance_filter')
        # Listen to 'distance_raw'
        self.ym_subscription = self.create_subscription(
            Float32, 'distance_raw', self.filter_callback,
            10)
        # Publish on 'distance_filtered'
        self.ym_publisher = self.create_publisher(Float32, '
            distance_filtered', 10)

        # Filter State
        self.ym_data_buffer = [0.0, 0.0, 0.0]
        self.ym_min_val = 3.0
        self.ym_max_val = 380.0

    def filter_callback(self, msg):
        ym_raw_val = msg.data

        # 1. Range Validation
        if ym_raw_val < self.ym_min_val or ym_raw_val > self.
            ym_max_val:
            out_msg = Float32()
            out_msg.data = 0.0
            self.ym_publisher.publish(out_msg)
            self.get_logger().warn(f'Out of range: {ym_raw_val
                :.2f} cm')
            return

        # 2. Median Filter (Depth 3)
        self.ym_data_buffer.pop(0)
        self.ym_data_buffer.append(ym_raw_val)

        sorted_data = sorted(self.ym_data_buffer)
        ym_median_val = sorted_data[1]

        # 3. Publish
        out_msg = Float32()
        out_msg.data = ym_median_val
        self.ym_publisher.publish(out_msg)
```

```
self.get_logger().info(f'Filtered Median: {
    ym_median_val:.2f} cm')

def main(args=None):
    rclpy.init(args=args)
    ym_node = DistanceFilterNode()
    try:
        rclpy.spin(ym_node)
    except KeyboardInterrupt:
        pass
    finally:
        ym_node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

0

VIII. DATA SANITIZATION AND PID INTEGRATION (SYBREN)

However, during the experiments, some of the ultrasonic readings could be incorrect due to acoustic reflection and noise in the environment. To increase the reliability of the readings coming from the sensor, the median filter was applied using the three-sample sliding window. The N value equal to 3 was chosen since it was the optimal choice in terms of noise cancellation and lagging. With the increase of the number of samples in the window, the delay within the control loop would be higher, and, at the same time, with the window size of $N = 3$, almost all outlier readings were eliminated.

Feeding in the smooth data into the PID was the last step. The node computes the error by setting a 30 cm threshold. Once the distance becomes less than 30 cm, the proportional and derivative computation comes up with a quick angular velocity instruction (`Twist.angular.z`) that results in the robot rotating sharply until it clears its forward vector, when it resumes its normal forward velocity (`Twist.linear.x = 0.5`).

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
from geometry_msgs.msg import Twist

class PIDDistanceController(Node):
    def __init__(self):
        super().__init__('pid_distance_controller')

        # Directly command the motor node gateway
        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel',
            10)

        self.ultra_sub = self.create_subscription(Float32, '
            distance_filtered', self.ultra_callback, 10)
        self.lidar_sub = self.create_subscription(Float32, '//
            path_blocked', self.lidar_callback, 10)

        # State memory storage
        self.lidar_blocked = False
        self.is_escaping = False
        self.latest_ultra_dist = 20.0 # Default safe setpoint
            distance

        # PID tracking configurations
        self.setpoint = 20.0
        self.kp = 0.22
        self.ki = 0.001
        self.kd = 0.02
        self.integral = 0.0
        self.prev_error = 0.0
        self.max_integral = 5.0

        self.prev_time = self.get_clock().now()
```

```

# FIXED: A dedicated execution timer running at
# exactly 25Hz (every 0.04s)
# This guarantees a smooth, continuous stream of
# commands to the motors with no drops.
self.control_timer = self.create_timer(0.04, self.
    control_loop)

self.get_logger().info("Unified Timer-Driven Brain
    Online. Lag protection active.")

def lidar_callback(self, msg):
    # Quietly record state in background
    self.lidar_blocked = (msg.data == 1.0)

def ultra_callback(self, msg):
    # Quietly record state in background
    self.latest_ultra_dist = msg.data

def control_loop(self):
    # Pull asynchronous snapshots safely
    ultra_dist = self.latest_ultra_dist
    if ultra_dist <= 0.0:
        return

    current_time = self.get_clock().now()
    dt = (current_time - self.prev_time).nanoseconds / 1e9
    self.prev_time = current_time

    # Safeguard for anomalous timing packets
    if dt <= 0.001:
        dt = 0.04

    # --- STATE MACHINE LOGIC ---
    # Trigger escape rotation if Lidar sees a block OR
    # ultrasonic gets dangerously close
    if self.lidar_blocked or ultra_dist < 15.0:
        self.is_escaping = True

    # Recovery Condition: Stop rotating only when BOTH
    # sensors see a completely clear path ahead
    if self.is_escaping:
        if not self.lidar_blocked and ultra_dist > 40.0:
            self.is_escaping = False
            self.integral = 0.0 # Clear out accumulated
            winding errors
            self.prev_error = 0.0

    twist = Twist()

    if self.is_escaping:
        # Execute a stable high-torque pivot on the spot
        twist.linear.x = 0.0
        twist.angular.z = -0.65
    else:
        # Path is wide open! Run your standard forward/
        # backward tracking PID loop
        error = self.setpoint - ultra_dist
        self.integral += error * dt
        self.integral = max(min(self.integral, self.
            max_integral), -self.max_integral)
        derivative = (error - self.prev_error) / dt

        output = (self.kp * error) + (self.ki * self.
            integral) + (self.kd * derivative)
        cmd_speed = max(min(-output, 0.8), -0.8)

        twist.linear.x = cmd_speed
        twist.angular.z = 0.0
        self.prev_error = error

    # ALWAYS publish a command every single loop execution
    # cycle to prevent motor stalling
    self.cmd_pub.publish(twist)

def main(args=None):
    rclpy.init(args=args)
    node = PIDDistanceController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:

```

```

        node.cmd_pub.publish(Twist())
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

This node receives the filtered distance and compares it with a desired setpoint of 20 cm. A PID controller calculates the required forward or backward velocity needed to maintain this distance. The controller includes proportional, integral, and derivative terms, as well as an anti-windup mechanism to prevent excessive accumulation of the integral term. The calculated linear velocity is published as a Twist message on the /cmd_vel_distance topic.

IX. LiDAR INTEGRATION AND OPTIMIZATION (SYBREN)

Integrating the LiDAR was the most technically demanding aspect of the project. Simply acquiring the driver required reaching out to our professors. Once the sensor was spinning, udev assignment rules were required to stop Linux from shuffling the USB port assignments on every boot.

The primary bottleneck was the sheer volume of data. The sensor_msgs/LaserScan message contains an array of 360 floating-point numbers published at 10 Hz. Attempting to parse this in Python choked the Pi's CPU, creating a 2-second delay between the robot seeing an obstacle and actually stopping.

In order to achieve this, I took out most of the fat from the computation process. I cut off the input data array to leave only 90 degrees worth of data, which were the front side of the robot. I changed the QoS profile to BEST_EFFORT in order to prevent the network layer from storing old laser scans. Finally, I replaced the spatial mapping process by just sending either a true or false value based on whether or not there are more than 20 points in the 90-degree front angle less than 40 cm away from the robot.

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from std_msgs.msg import Float32
from rclpy.qos import QoSProfile, ReliabilityPolicy,
    HistoryPolicy

class LidarSteeringNode(Node):
    def __init__(self):
        super().__init__('lidar_steering_node')

        qos = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,
            history=HistoryPolicy.KEEP_LAST,
            depth=10
        )

        self.subscription = self.create_subscription(LaserScan,
            '/scan', self.lidar_callback, qos)
        # Publishes a 1.0 if front is blocked, 0.0 if clear
        self.block_pub = self.create_publisher(Float32, '/
            path_blocked', 10)

        # 50cm detection threshold for structural blockages
        self.BLOCKED_THRESHOLD = 0.50

    def lidar_callback(self, msg):
        if not msg.ranges:
            return

        num_readings = len(msg.ranges)
        if num_readings < 360:

```



```

        return

    # Sweeping a wide 90-degree front cone window (135 to
    # 225 degrees)
    front_cone = msg.ranges[135:225]
    valid_front = [r for r in front_cone if msg.range_min
        < r < msg.range_max]
    min_front = min(valid_front) if valid_front else float(
        'inf')

    out_msg = Float32()
    if min_front < self.BLOCKED_THRESHOLD:
        out_msg.data = 1.0 # Path is blocked!
    else:
        out_msg.data = 0.0 # Path is clear!

    self.block_pub.publish(out_msg)

def main(args=None):
    rclpy.init(args=args)
    node = LidarSteeringNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

This node is intended to process data from the LiDAR sensor to detect and track the closest object within a 30° field of view in front of the robot. The angle of the detected object is compared with the desired forward direction, and a proportional controller generates an angular velocity command that steers the robot towards the object. This command is published on the /cmd_vel_angle topic. Due to hardware issues with the LiDAR, this node has not yet been fully tested or verified.

X. SYSTEM INTEGRATION

After all individual components had been tested successfully, they were integrated into one ROS 2 application. Communication between the nodes was verified using ROS 2 command-line tools and the RQT Graph, ensuring that each topic was correctly connected.

This whole system operated on a simple operational pattern. The ultrasonic sensor kept measuring the distance ahead of the robot. The controller analyzed the measurement and determined whether the robot needed to stop or not because of any obstacles. In case of no obstacles in the safe distance ahead, the robot kept moving ahead due to the operation of the motor node. But in case of an obstacle ahead, the robot stopped and turned around and then moved again.

Several complete system tests were performed indoors. After minor adjustments to the sensor thresholds and motor speeds, the robot was able to navigate reliably without colliding with obstacles.

XI. RESULT

The fully integrated system underwent rigorous physical testing across various indoor environments (carpet, tile, tight corridors).

The median filter proved highly effective; plotting the output of /distance_raw against /distance_filtered showed a

complete elimination of high-frequency noise spikes, resulting in incredibly smooth motor responses from the PID controller. The software-PWM motor node handled torque adjustments well, though slight differences in the manufacturing of the two DC motors meant the robot drifted slightly to the left during long straightaways—a common issue in differential drives lacking wheel encoders.

When considering the robot's primary task of autonomous obstacle avoidance, it proved to be a complete success. The robot was capable of navigating hallways autonomously. When the robot got near the wall or leg of a person, the ultrasonic trigger kicked in as expected. This resulted in the PID control reducing the linear velocity and increasing the angular velocity, making a turn.

Even though the LiDAR's hardware constraints did not allow us to perform the task of dynamic object following, the boolean flag optimization we performed demonstrated that dense data could indeed be processed and applied within our embedded system without causing any crashes.

Discussion and Outlook

This project was a resounding success in establishing a robust, modular autonomous framework, even if our final operational mode diverged from our initial proposal.

The greatest architectural success of this project is our strict adherence to ROS 2 modularity. In case of high latency problems in the LiDAR subsystem, because of the modularity of DDS, everything else in the robot worked correctly. It was possible to skip the LiDAR node altogether and navigate only with ultrasonic input data. The ability to use new modules without any major changes in the code of the core functionality is the evidence of high scalability of our solution.

The problems that we encountered were very typical for practical embedded engineering. Devices don't talk to each other automatically, permissions of Linux are always an issue, and computing power is not infinite and needs to be fought for viciously. The shift from coding to practical application has shown us that mechanical resistance, acoustic interference, and voltage drops have as much impact on robotics as any algorithm.

Moving into the future, there is great potential for further development based on what was accomplished in this project. The very next logical step is to use hardware encoders on the DC motors in order to have closed loop odometry and the capability of moving perfectly straight lines. Having achieved that, it becomes possible to again activate the LiDAR hardware and use SLAM algorithm.

XII. TEAM EXPERIENCE AND CONTRIBUTIONS

This project was a huge step into the unknown territory for all of us. Our group moved from theory learned in classrooms to a very complicated practical engineering task. We needed to master Linux system management, embedded programming in Python, C++ compilations, and hardware issues altogether. It became clear that our way to success would be based mainly on using the documentation available (for example, ROS 2 Humble

tutorials) and the knowledge from the Internet communities (Stack Overflow, GitHub).

Mohammedmed and Ziad: Acted as the core systems engineer. He managed the volatile Linux/ROS 2 environment setup, ensured network stability, and programmed the low-level GPIO interface, including the critical safety shutdown protocols for the L298N driver.

Zaid: Led the mechanical engineering effort. He designed the spatial layout of the robot in SolidWorks, managed the 3D printing pipeline, and iterated on physical tolerances to ensure the electrical hardware was housed securely and safely.

Sybren: Served as the core control logic programmer. He built the foundational mock-nodes in C++, translated them to Python, engineered the median-filter mathematical logic, and tuned the PID controller responsible for the robot's smooth evasive maneuvers.

Specialized in sensor fusion and hardware optimization. I spearheaded the LiDAR integration, untangled the complex Linux USB assignments, and optimized the Python parsing algorithms to prevent CPU thermal throttling, successfully converting a dense hardware data stream into usable ROS 2 topics.

XIII. MAIN LEARNING OUTCOMES

In addition to the practical achievement of building a functional robot, this project also changed our perspective on how to design engineering projects. We learned about the experience of creating everything within the scope of Mechatronics from the tolerance of the plastic chassis to high level distributed software.

We have realized that when something does not work right in an embedded system, it is rarely a simple problem. A robot that will not turn may be a logical error in Python, or an executable that has been cached in the ROS workspace, or the robot may have poor grounding on its breadboard, or a broken plastic gear in its motor housing.

Most importantly, we were able to understand the importance of agile engineering. Rather than giving up when the LiDAR tracking was not feasible within our hardware limitations, we were able to evaluate what works and come up with an alternate approach to deliver a very useful obstacle avoidance system. This capability under difficult conditions is one of the most important lessons that we have learned this semester.

REFERENCES

- [1] Open Robotics, "ROS 2 Humble Hawksbill Documentation," 2022. [Online]. Available: <https://docs.ros.org/en/humble/index.html>.
- [2] Raspberry Pi Foundation, "Raspberry Pi 4 Model B Datasheet and Documentation," 2023. [Online]. Available: <https://www.raspberrypi.com/documentation/>.
- [3] Canonical Ltd., "Ubuntu 22.04 LTS (Jammy Jellyfish) Release Notes," 2022. [Online]. Available: <https://ubuntu.com/documentation>.
- [4] SparkFun Electronics, "HC-SR04 Ultrasonic Distance Sensor Datasheet," Revision 1.4.
- [5] Slamtec, "RPLiDAR A1/A2 Development Kit User Manual and ROS Integration Guide," 2021.
- [6] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*. Berlin, Germany: Springer, 2011.
- [7] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.